Highlights

GAMA-RCA: Graph-aware, Multimodal, Abductive Root Cause Analysis for Microservices with a Computed Belief and a Bounded Language Model

Achmad Solaeman, Nur Rohman Rosyid, Ardhi Wicaksono, Prima Asmara Sejati, Hamzah

- The diagnosis is a computed posterior over (entity, fault type), not text written by a language model.
- Every signal is calibrated against the incident's own history and peers; no labels and no training are used.
- The statistical core reaches 62.9 Final on RCA100 on a CPU, 20 points above an LLM given the same evidence.
- A larger LLM does not decide better (both backbones lower the score), but it explains far better.
- All evaluation runs, hypotheses and amendments were pre-registered and are released with every prompt and output.

GAMA-RCA: Graph-aware, Multimodal, Abductive Root Cause Analysis for Microservices with a Computed Belief and a Bounded Language Model

Achmad Solaeman[a,*], Nur Rohman Rosyid[a], Ardhi Wicaksono[a], Prima Asmara Sejati[a], Hamzah[b]

[a] *Department of Electrical Engineering and Informatics, Vocational College, Universitas Gadjah Mada, Yogyakarta, Indonesia*
[b] *PT Media Pangan Kreatif Indonesia, Indonesia*

Abstract

Root cause analysis (RCA) of microservice failures must name the faulty component, its fault mechanism and the propagation path from metrics, logs, traces and events. Agents built on large language models (LLMs) reason over this telemetry at a cost of up to millions of tokens per incident, with conclusions that are hard to audit. We present GAMA-RCA, a label-free method in which the diagnosis is *computed*. Every signal is scored against the incident's own history and its peers and calibrated on pseudo-windows cut from that history; the observations update a Bayesian belief over (entity, fault type) hypotheses with fingerprint-based likelihoods, topology explain-away, a per-incident propagation strength estimated by expectation–maximisation, and temporal precedence. A two-stage decision names the root entity and its type, and a deterministic chain cites the evidence of every step. An optional, bounded LLM may read log text, choose a type from the statistical shortlist and narrate the chain. On the 103 cases of RCA100, the statistical core reaches a Final score of 62.9 [95% CI 57.4, 68.5] on a CPU, 20.2 points above an LLM that decides from the same evidence ($p = 2.8 \times 10^{-10}$). In a pre-registered study, the bounded LLM lowers the score by 3.7 (Qwen2.5-7B) and 3.5 points (gpt-oss-120b): scale does not improve the decision, but it improves the explanation, raising the narrator's evidence-checkpoint rate

*Corresponding author.
Email address: achmad.solaeman@ugm.ac.id (Achmad Solaeman)

from 0.107 to 0.284, close to the computed chain (0.307). **[Pending: zero-shot AIOps2025 result.]**

Keywords: root cause analysis, microservices, AIOps, Bayesian inference, observability, large language models, explainability

1. Introduction

A microservice application is a graph of independently deployed services that call each other over the network and share hosts, caches and databases. When one component fails, the failure crosses component boundaries: an exhausted CPU on a node slows every service it hosts, a failing payment service returns errors to the checkout service that calls it, and the operator sees alerts far from the cause [1, 2]. Root cause analysis (RCA) must therefore answer three questions from the incident's telemetry—metrics, logs, traces, Kubernetes events and the service topology: *which* entity is the root cause, *what* fault mechanism acts there, and *how* the fault propagated to the symptoms [3, 4]. Recent benchmarks score all three. RCA100 [5] grades entity localisation, fault identification over 28 fault types and the reasoning process along a typed causal chain with numeric evidence checkpoints.

Two families of methods address the task. Statistical and causal methods rank candidate components from metric changes, trace structure or learned causal graphs [6, 7, 8, 9, 10, 11]. They are cheap and reproducible, but they usually return a ranked list of components without a fault type or an explanation. Agents built on large language models (LLMs) read the telemetry through tools and write a diagnosis in natural language [12, 13, 14, 15, 16]. They produce types and explanations, but a trajectory-level study of such agents finds systematic reasoning failures—untested baselines, ignored timing, reversed caller / callee semantics, anchoring on the first hypothesis [17, 18]—and the strongest agents consume on the order of 10^6 tokens per incident [17, 19]. Agents that maintain an explicit belief state [19, 20] still ask the LLM to *write* that belief.

This paper takes the opposite position: the belief over root causes can be *computed* from calibrated evidence, and the language model, if used at all, should be bounded to the parts that need language. We present GAMA-RCA (Graph-aware, Multimodal, Abductive RCA). Every continuous signal of every entity is scored against the incident's own history and its peers, and the score is turned into a p-value by an empirical null cut from that

history (pseudo-windows); events, status codes, failing spans and log text become categorical observations. A Bayesian model over (entity, fault type) hypotheses scores these observations with fingerprints that state, for each fault type, which signals are key, supporting, possible consequences or contradictions. Symptoms that a hypothesis can explain along the topology are explained away; the propagation strength is a per-incident nuisance parameter estimated by expectation–maximisation; and a root may not start after the symptoms it explains. The decision is two-stage (entity, then type), and the causal chain is computed along the topology, citing the window values of every step. An optional LLM may read log text chosen by expected information gain, choose a type from the statistical shortlist with a verbatim quote, and narrate the chain.

We evaluate GAMA-RCA on the 103 cases of RCA100 under a pre-registered protocol (hypotheses, statistics and every amendment recorded before the corresponding run) and release every prompt and output. The results support three statements. First, the statistical core alone reaches a Final score of 62.9, on a CPU and without labels, 20.2 points above an LLM that decides from the same calibrated evidence and 44.1 points above an LLM that reads the alert only. Second, the bounded LLM lowers the decision score by 3.7 (Qwen2.5-7B) and 3.5 points (gpt-oss-120b): when the telemetry cannot separate two fault types, the LLM tends to pick the more specific one and is more often wrong than right. Third, scale improves the *explanation*: the narrator of the larger model hits 2.7 times as many step-anchored evidence checkpoints as the 7B narrator, close to the computed chain. We then transfer the method zero-shot to AIOps2025 [5], a different system with different fault types, changing only dataset knowledge (an adapter and a fingerprint table written from the published fault definitions).

Contributions.

1. **A computed abductive belief for RCA.** A label-free likelihood model over (entity, fault type) hypotheses with fingerprint roles, topology explain-away, per-incident propagation strength (EM) and temporal precedence (Section 4).
2. **Calibrated multimodal evidence.** Shift scores against the incident's own history and peers, calibrated on pseudo-windows, together with trace-origin attribution, direction-aware load explain-away, mechanism-naming text and cross-modal corroboration as separable modules (M1–M6).
3. **A pre-registered study of what an LLM adds.** Seven arms on the

same 103 cases, from an LLM that sees only the alert to the computed belief alone, two backbones (7B local, 120B through an API) and hypotheses fixed in advance (Sections 5 and 6): the LLM does not improve the decision at either scale, while scale improves the explanation.

4. **A zero-shot transfer protocol** to an unseen system, pre-registered with an implementation audit, and an open implementation with equivalence and synthetic tests (Section 6.5).

The paper is organised as follows. Section 2 reviews related work, Section 3 states the problem and the metrics, Section 4 describes the method, Section 5 the experimental protocol, Section 6 the results, Section 7 discusses implications and threats to validity, and Section 8 concludes.

2. Related work

Statistical and causal RCA. Graph-based methods propagate anomaly scores over the call graph [6, 21]; MicroHECL distinguishes performance, reliability and traffic anomalies by their direction of propagation [22]. Causal methods learn a graph from metric time series or recognise the intervention that a fault represents [7, 9, 23]. BARO detects multivariate change points with a Bayesian online model and ranks the changed services [8]; ϵ-Diagnosis tests small windows against history [24]. Trace-based methods locate the deepest failing or slowest span [25, 26, 27]. Multimodal methods fuse metrics, logs and traces into shared representations or event graphs, often with training [28, 29, 10, 11]. These methods share our preference for computation over generation, but most output a ranking of components; fault types and an evidence-cited causal chain are outside their scope, and the evidence from different modalities is usually fused before any reasoning about mechanisms. GAMA-RCA keeps the statistical tests of this line of work but places them in a likelihood over typed hypotheses, so that the same model names the entity, the mechanism and the evidence.

Probabilistic diagnosis. Explaining away [30] and noisy propagation over dependency graphs [31] are the classical foundations of our belief model; active probing with information gain [32, 33] motivates the text sensor. Calibration of test scores against an empirical null [34] and conformal calibration [35] motivate our pseudo-window p-values. Compared with these works, GAMA-RCA instantiates the hypothesis space with fault *types* from a benchmark taxonomy, derives likelihoods from fingerprint roles instead of training, and estimates the propagation strength per incident.

LLM-based RCA. RCACopilot predicts incident categories from collected diagnostic information [12]; RCAgent and ReAct-style agents call tools in an open loop [13, 36, 37]; Flow-of-Action and mABC organise several agents around procedures or votes [15, 16]; GALA refines a statistical ranking with agentic re-ranking [38]. OpenRCA, RCAEval, AIOpsLab and RCA100 turn the task into benchmarks [14, 39, 40, 5]; OpenRCA 2.0 adds causal process supervision [41]. Lu et al. [17] analyse agent trajectories on RCA100 and AIOps2025 and catalogue omission, misreading and generation failures; Riddell et al. [18] report stalled and biased reasoning. Closest to our bounded LLM are the combination of an LLM with a small classifier [42], confidence estimation for root causing [43] and grounding / verification of generated claims [44, 45].

Belief states and abduction with LLMs. Long-horizon agents that maintain an explicit belief state, or a graph of hypothesis states, improve over memory-only agents on abductive tasks including RCA100 [19, 20]; planning under uncertainty with LLM-generated hypotheses follows the POMDP view [46, 47]. In these systems the LLM writes and updates the belief. In GAMA-RCA the belief is a posterior computed from calibrated likelihoods; the LLM can only contribute a likelihood term (sensor) or a choice among statistically ranked options (typing).

Table 1 summarises the positioning, and Table C.10 (Appendix C) maps the failure modes catalogued by Lu et al. [17] to the mechanism of GAMA-RCA that rules each out by construction.

Table 1: Positioning of GAMA-RCA among RCA approaches.

Approach	Decision made by	Output	Evidence calibration	Labels
Statistical / causal [6, 7, 8]	scores / causal graph	ranked components	per method	no
Multimodal learned [29, 28]	trained model	component, type	learned	yes
LLM agents [13, 14, 15]	LLM	free text, chain	none (LLM reads raw data)	no
Belief-state agents [19]	LLM-written belief	entity, type, chain	none	no
GAMA-RCA (this work)	**computed posterior**	**entity, type, cited chain**	**history, peers, pseudo-windows**	**no**

3. Problem statement and evaluation metrics

Incident. An incident (case) consists of an alert window $W = [w_0, w_1]$ with an alert text and, when available, the alerted service and operation; a topology graph G whose nodes are services, operations, pods and hosts (nodes) and whose edges are *calls* and *hosting* relations; and the telemetry of the window and of the time before it: metric series per entity, log lines, spans of distributed traces and Kubernetes events. No label is available at diagnosis time.

Output. A diagnosis is a triple (e^*, τ^*, C): the root-cause entity e^* (a service or a node), its fault type τ^* from a fixed taxonomy $\mathcal{T}$, and a causal chain C of steps *cause* $\rightarrow$ *propagation* $\rightarrow$ *impact*, each naming an entity and the evidence (signal and value) that supports it. Following the dependability vocabulary [1], the root cause is the component where the fault is activated; the chain records how the resulting errors and load cross component boundaries to the alerted operation.

Metrics. We use the three pillars of the RCA100 protocol [5]. *Entity localisation* (EL) is 1 for the true entity and 0.5^d for an entity at topological distance d. *Fault identification* (FI) is 1 for the exact type and 0 otherwise. The *reasoning process* (RP) is the mean of (i) the share of ground-truth chain nodes named by the predicted chain and (ii) the share of ground-truth evidence checkpoints satisfied, where a checkpoint is a signal with a numeric constraint (e.g. `error_count` ≥ 8829) that the cited evidence must meet. The Final score is

$$\text{Final} = 0.4\,\text{EL} + 0.3\,\text{FI} + 0.3\,\text{RP}, \tag{1}$$

reported on a 0–100 scale. In the headline RP the checkpoints are searched in the evidence the system retrieved; this rewards systems that return evidence items and gives zero to a chain written by an LLM without them. We therefore also report a *step-anchored* variant ($\text{Final}_{\text{step}}$), in which a checkpoint counts only if the chain step that names the checkpoint's target entity cites a value meeting the constraint. An official scoring script was not part of the RCA100 release we used; our scorer is a re-implementation of the published protocol and is released with the code. We also count the cases with the exact entity, the exact type and both (joint).

4. Method

Figure 1 shows the pipeline. Three principles guided every design decision. **P1, calibrate, do not threshold:** every observation is scored against the incident's own history and peers, and likelihoods use rates rather than cut-offs. **P2, causes precede and propagate directionally:** errors travel up the call graph to callers and load travels down to callees; a symptom the root can explain is not evidence against it, and load that came from upstream is not evidence for a downstream root. **P3, the LLM reads text and chooses among ranked options:** the decision and its evidence are computed. The method has a core (stages 1–3 and the chain) and seven switchable modules: M1–M6 form the statistical core (GAMA), and M7 is the bounded LLM. All constants are listed in Table A.8 (Appendix A); none was fitted to labels.

4.1. Stage 1: calibrated evidence

Signals. Metric series are mapped to signal families by their names (CPU, memory, garbage collection, threads, disk, throttling, request count, latency,

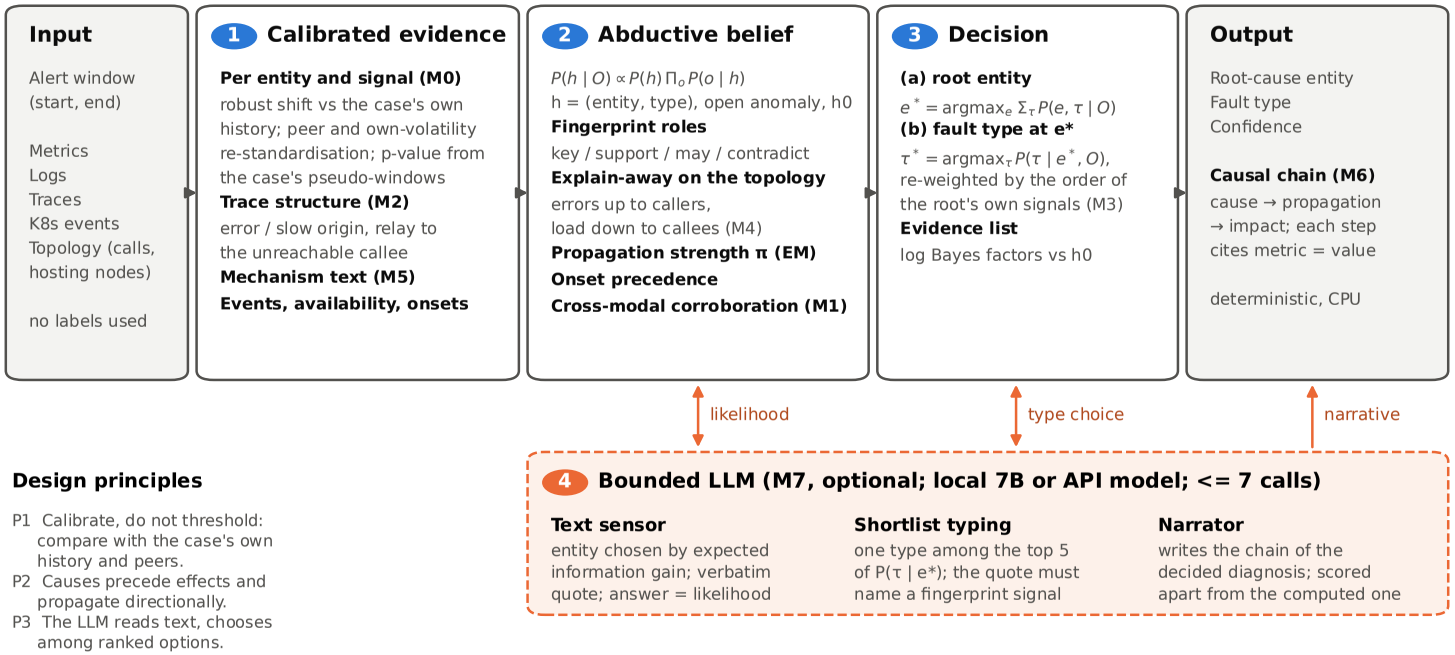


Figure 1: GAMA-RCA. Stage 1 turns telemetry into calibrated binary observations; stage 2 computes a posterior over (entity, fault type) hypotheses; stage 3 decides the root entity and then its type; the chain (M6) cites window values along the topology path. The bounded LLM (stage 4, optional) contributes a likelihood term (sensor), a choice among the statistical shortlist (typing) and a narrative that is scored separately.

errors, restarts, replicas / availability); configuration constants (limits, initial sizes) are excluded. For a service the signals are, for instance, `svc_cpu_rise`, `svc_latency_rise` or `svc_request_drop`; for a node `node_cpu_rise`, `node_mem_rise`, `node_disk_rise` and `node_ready_drop`. Each signal has a direction $d \in \{+1, -1\}$ (rise or drop).

Robust shift. Let x be a series of entity e and $[a, b)$ a tested window of length $L = b - a$. Its history is $\mathcal{H} = [a - BL, a)$ with $B = 4$. The directional robust shift is

$$z = d \cdot \frac{\operatorname{med}_{t\in[a,b)} x_t - \operatorname{med}_{t\in\mathcal{H}} x_t}{\sigma_{\mathcal{H}}}, \qquad \sigma_{\mathcal{H}} = \max\Big(1.4826\ \mathrm{MAD}_{\mathcal{H}}(x),\ 0.05\,|\operatorname{med}_{\mathcal{H}} x|,\ \epsilon\Big), \tag{2}$$

computed only when the history covers at least 80% of its expected samples and the window at least half of them. Error counts are summed in 30 s bins (zero-filled) with a scale floor of one event per bin. The score of a signal is the largest z over the entity's series of that family.

Pseudo-windows and calibration. A shift score is not a probability: series drift and are autocorrelated, so the window and its history are not exchangeable, and a plain rank test flagged about 30% of fault-free windows in our development runs. We therefore compute the same score on $K = 5$ pseudo-windows $W_j = [w_0 - jL,\ w_0 - (j-1)L)$ cut from the incident's own pre-alert data, each against its own history. Two re-standardisations are applied identically to real and pseudo scores. *Own volatility*: each entity's score is re-centred and re-scaled by the median and MAD of its pseudo scores (leave-one-out for the pseudo-windows, floor 1), so a chronically noisy entity does not outrank one that changed only now. *Peers*: in every window the score is re-standardised against the scores of the other entities of the same kind (at least three peers, floor 1), which rejects a load step or a drift that moves all of them. The calibrated p-value of the real window is the conformal rank of its score among the pseudo scores of all entities of the same kind and signal in the incident [34, 35],

$$p = \frac{1 + \#\{z' \in \mathcal{N}_{k,s} : z' \geq z\}}{|\mathcal{N}_{k,s}| + 1}, \tag{3}$$

pooled over the entity kind when fewer than 20 pseudo scores exist. The signal *holds* when $p \leq \alpha = 0.05$. The *onset* of a holding signal is the first sample whose shift against a history ending one window earlier reaches 3 for two consecutive samples.

Categorical observations. Kubernetes events (OOM kill, crash loop, failed probe, scheduling failure, scale-down, restart, eviction, node not ready) count only when their increments inside the window exceed their own long-run rate (Poisson test). Availability below the desired replica count is a state. Failing spans contribute their HTTP / gRPC status class and the mechanism named by their message (null pointer, rate limit, DNS, connection refused, timeout, Redis, out of memory, GC, thread pool, queue, load balancer, network policy, SQL, cache); failing or slow dependency spans (database, cache, queue) are separate signals. The background rate of a categorical observation is the share of entities of the same level showing it in the incident, shrunk towards α with a Beta prior worth ten pseudo-entities. Each incident yields on the order of 10^3 binary observations $o = (e, s, \mathit{holds}, \mathit{onset})$.

4.2. Stage 2: abductive belief

Hypotheses. For every service e and fault type τ of service level, and every node and type of node level, $h = (e, \tau)$ is a hypothesis; each entity also has an open hypothesis "anomaly of unspecified type", and h_0 denotes no fault. A hypothesis is *testable* only if at least one of its key signals was computed. The posterior is

$$\Pr(h \mid O) \;\propto\; \Pr(h) \prod_{o \in O} \Pr(o \mid h), \tag{4}$$

with equal prior mass per entity, split among its hypotheses (the two fallback types of the taxonomy, *httpError5xx* and *codeDefect*, at half weight), and $\Pr(h_0) = 0.1$.

Fingerprint roles. For each type, a fingerprint written from the taxonomy definitions assigns roles to signals: *key* (at least one must hold), *support*, *may* (a possible consequence, whose absence is uninformative) and *contradiction*. For example, *cpuFullLoad* has the key `svc_cpu_rise`, the support `svc_latency_rise` and the contradiction `svc_request_rise` (a CPU rise caused by more traffic is a traffic fault). For an observation at the hypothesised root,

$$\Pr(o \text{ holds} \mid h) = \begin{cases} q_h & \text{key}, \quad q_h = 1 - (1 - \theta_K)^{1/n_h} \\ \theta_U & \text{support or may} \\ \alpha_C & \text{contradiction} \\ \beta_s & \text{otherwise (background)}, \end{cases} \tag{5}$$

where n_h is the number of testable keys, so that the probability that some key holds is θ_K. Symptoms expected at any root (latency, errors, failing spans) have probability $\max(\theta_P, \beta_s)$, and the absence of a *may* signal leaves the likelihood at its h_0 value.

Explaining away along the topology. Let $\mathrm{Expl}(e)$ be the entities whose fault can show as a symptom at e: the downstream closure of e in the call graph and the nodes hosting e or any of them. A symptom at $e \neq e_h$ with $e_h \in \mathrm{Expl}(e)$ is a *propagated* observation: it holds with probability π, the per-incident propagation strength. A node CPU rise is explained by a root whose fault loads its host. All other observations away from the root keep their background rates, so a hypothesis that explains many symptoms is not rewarded for symptoms that do not occur: a wide hypothesis pays $\log(1-\pi)$ for every reachable entity that stays quiet.

Propagation strength by EM. Write $\log \Pr(O \mid h) = \ell_0(h) + n^+(h) \log \pi + n^-(h) \log(1-\pi)$, where $n^{\pm}(h)$ count the propagated symptoms that hold or are absent. Expectation–maximisation [48] alternates the posterior $w_h \propto \Pr(h) \Pr(O \mid h)$ and

$$\pi \leftarrow \frac{\sum_h w_h \, n^+(h)}{\sum_h w_h \left(n^+(h) + n^-(h) \right)}, \tag{6}$$

clipped to $[0.06, 0.95]$, until π changes by less than 10^{-4} (at most 30 iterations).

Temporal precedence. A root does not start after what it causes. For h with root e_h whose onset t_{e_h} is measured, every explained entity with an onset t_e contributes $\log(\theta_T/0.5)$ if $t_e \geq t_{e_h} - 60\,\mathrm{s}$ and $\log((1-\theta_T)/0.5)$ otherwise; a root without a measured onset is neutral. Onsets of load explained from upstream (M4) are ignored.

4.3. Modules

M1 Cross-modal corroboration. A real fault tends to leave aligned evidence in several modalities, whereas the noise of a volatile entity tends to stay in one [28, 5]. Let $k(h)$ be the number of modalities (metrics, logs, traces, events) among the holding key, support or may signals at the root. If $k(h) \geq 2$ the hypothesis gains $\log(\theta_M/\beta_M)$, where β_M is the share of the incident's specific hypotheses with such corroboration, shrunk to a floor of 0.1; absence is uninformative.

M2 Trace-origin attribution. In every failing trace the error origins are the deepest failing spans, none of whose children fails; a service is an error origin when it originates at least 30% of the failing traces and at least ten of them. In every trace the service with the largest self time (span duration minus the time covered by its children) is where the time is spent; a service is a slow origin when its share among the slowest 10% of the traces of a root operation exceeds its share among the others by at least 0.2, in at least five traces. A failing client span that names an unreachable callee relays the origin to that callee, past the reporting caller [25]. An origin is own evidence of its entity (and of a node that hosts at least two origin services), and the messages and codes of origin spans enter as categorical observations.

M3 Mechanism typing. The type stage uses three rules. A node CPU rise is explained by any root whose own CPU rises. Error origins are expected only by types whose fingerprint names an error signal. Within the root, the cause moves before its consequences: for a type whose key is a resource signal (CPU, memory, GC, threads), θ_T applies if that key is among the earliest-onset resource signals of the root, 0.5 if it holds without a measured onset and $1 - \theta_T$ otherwise. This term re-weights the types of the decided root; it does not move the root. Exact ties are broken general-before-special by the fingerprint order.

M4 Direction-aware load explain-away. A request-count rise at a callee whose caller's requests rise too is load that came from upstream [22]: its request, latency and CPU rises, and the CPU rise of the nodes hosting it, receive the background θ_P for every hypothesis.

M5 Mechanism-naming text. Span exception text and gRPC status codes are read as mechanism evidence. A log text family becomes an observation only if it is new in the window (Poisson test against the same length of history) and names a mechanism; symptom text ("timeout", "refused") is excluded, because almost any fault produces it.

M6 Evidence-cited chain. The chain follows a shortest call path from the decided root to the alerted service (for a node root, from the hosted service closest to it) and ends at the alerted operation. Each step cites up to four window values of its own target (golden signals: requests, errors, latency for services; CPU, memory, disk for nodes [49]); the cause step first cites the readings behind the root's holding evidence.

4.4. Stage 3: decision

The decision is two-stage. The root entity maximises the marginal posterior, $e^* = \arg\max_e \sum_\tau \Pr(e, \tau \mid O)$ (h_0 excluded, since an alert fired), and the type maximises the conditional posterior at that entity, $\tau^* = \arg\max_\tau \Pr(\tau \mid e^*, O)$, after the M3 re-weighting. When only the open anomaly is testable at e^*, the type with the strongest key evidence of its level is returned. The evidence list contains the observations whose log Bayes factor for the decided hypothesis against h_0 exceeds 0.5 in absolute value; it is the material the chain and the LLM cite. Algorithm 1 summarises stages 1–3.

Algorithm 1 GAMA on one incident (no labels, no training)

Require: window $[w_0, w_1]$, topology G, telemetry, fingerprints F, modules $\mathcal{M} \subseteq \{\mathsf{M1}, \ldots, \mathsf{M6}\}$

$O \leftarrow \emptyset$
for all entities e, signals s **do**
    $z_0, \ldots, z_K \leftarrow$ shifts of the real window and the K pseudo-windows (Eq. (2))
    re-standardise by own volatility and by peers; $p \leftarrow$ conformal rank (Eq. (3)); add $(e, s, p \le \alpha, \text{onset})$ to O
end for
add categorical observations (events, availability, codes, messages, dependency spans; M2, M5 if on)
$H \leftarrow$ testable hypotheses (e, τ), open anomalies, h_0
compute $\ell_0(h)$, $n^{\pm}(h)$ with roles (Eq. (5)), explain-away and M4; add M1 term
repeat ▷ EM for the propagation strength
    $w \leftarrow$ posterior with π; $\pi \leftarrow \sum w\, n^+ / \sum w\,(n^+ + n^-)$
until $|\Delta\pi| < 10^{-4}$
add the onset-precedence term; $e^* \leftarrow \arg\max_e \Pr(e \mid O)$; $\tau^* \leftarrow \arg\max_\tau \Pr(\tau \mid e^*, O)$ (M3)
return (e^*, τ^*), evidence list, chain along G (M6)

4.5. Stage 4: the bounded LLM (M7, optional)

The LLM has three bounded roles, each with a fixed budget, and never sees a label or a score.

- **Text sensor.** While no hypothesis reaches a posterior of 0.8, the entity whose text answer has the largest expected information gain (at least 0.05 bits) is chosen [33, 32], and the LLM reads its preprocessed text (log templates tested against their pre-window rate, failing-span messages,

events). It names a failure category from a fixed list and must quote the text verbatim; the answer enters the posterior as a likelihood term. At most five probes.

- **Shortlist typing.** The LLM chooses the type of the decided root among the five most probable types, shown with signature cards (the cross-modal evidence each type would leave). The choice is accepted only if the type is in the shortlist, the quoted evidence line (at least ten characters) appears verbatim in the input, and that line names a signal of the chosen type's fingerprint.
- **Narrator.** The LLM writes the causal chain of the decided diagnosis, citing values from the evidence. The scored chain stays the computed one; the narrative is scored separately (Section 6.3).

Decoding is greedy (temperature 0, seed 0), with at most seven calls per incident.

5. Experimental setup

5.1. Research questions

RQ1 How accurate is the statistical core of GAMA-RCA without any LLM, compared with LLMs that decide from the same evidence or from the alert alone?

RQ2 What does each module contribute (leave-one-module-out)?

RQ3 What does a bounded LLM add to the decision and to the explanation, and does a larger model change the answer?

RQ4 Does the method transfer zero-shot to an unseen system?

5.2. Benchmarks

RCA100 [5] contains 103 incidents injected into the OpenTelemetry Demo on a managed Kubernetes cluster, with metrics, logs, traces, Kubernetes events, alerts and a topology of services, operations, pods and nodes per case. Each case has a ground-truth root entity, one of 28 fault types in six groups (Table 2) and a typed causal chain with evidence checkpoints (661 in total, 99.5% with a numeric constraint). Fourteen cases are node faults and 89 service-level faults. **AIOps2025** [5] contains 400 incidents injected with Chaos Mesh [50] into a HipsterShop deployment with TiDB and Redis over 18 days; it has no topology file, 3 pods per service and 11 nodes, and is scored by its own protocol (Section 6.5). AIOps2025 was never used during the design of GAMA-RCA.

Table 2: Benchmarks. RCA100 fault groups with their number of cases.

	RCA100	AIOps2025
System	OpenTelemetry Demo, managed Kubernetes	HipsterShop + TiDB + Redis, Kubernetes
Cases	103	400
Modalities	metrics, logs, traces, events, alerts, topology	metrics, logs, traces (no topology, no events)
Fault types	28 in 6 groups	18 in 9 categories (Chaos Mesh injections)
Levels	service 89, node 14	service, pod, node
Scoring	EL, FI, RP (chain + checkpoints)	LA, TA, explainability, efficiency

RCA100 groups: application logic 38; cloud resource (node) 14; JVM runtime 16; K8s lifecycle 12; middleware & DB 13; resource & performance 10.

5.3. Compared arms

All arms receive the same incident input and none reads a label (Table 3). *Evidence→LLM* gives the LLM the same calibrated evidence tables that GAMA-RCA computes (per-entity signals, propagation, infrastructure and trace summaries) together with the taxonomy, and lets it decide entity, type and chain in one call; it isolates the value of the computed belief from the value of the evidence. *Belief→LLM* adds GAMA-RCA's posterior (top entities with their top types and probabilities) to the Evidence→LLM prompt and lets the LLM decide freely; it tests whether the belief helps an LLM that keeps full authority. *Direct LLM* sees only the alert. The leave-one-module-out arms remove one of M1–M6 from GAMA.

Backbones. Qwen2.5-7B-Instruct [51] runs locally (one NVIDIA T4, bfloat16, greedy decoding, at most 1,024 new tokens). gpt-oss-120b [52] (mixture of experts, 5.1B active parameters) is served through an OpenAI-compatible API with reasoning effort *medium*, temperature 0, seed 0 and at most 8,192 completion tokens (reasoning included). Prompts, parsers and stages are byte-identical across backbones. API decoding is not guaranteed deterministic; every call is therefore stored with its prompt, output and token counts, and no run was repeated. Both backbones were released before the RCA100

Table 3: Arms. All run on the 103 RCA100 cases with identical inputs; LLM settings are identical across arms of a backbone.

Arm	Who decides	LLM calls	Hardware
GAMA (statistical core, M1–M6)	computed posterior	0	CPU
GAMA −Mi (ablation)	computed posterior without module i	0	CPU
GAMA + LLM (M1–M7)	posterior; LLM as sensor, shortlist typing, narrator	≤ 7	GPU / API
Belief→LLM	LLM, given evidence and posterior	1	GPU / API
Evidence→LLM	LLM, given the calibrated evidence	1	GPU / API
Direct LLM	LLM, given the alert only	1	GPU

benchmark, and the LLM never receives an answer, a label or a score.

5.4. Statistics and pre-registration

We report means with 95% percentile bootstrap intervals (5,000 resamples, seed 0) [53]. Paired comparisons use the Wilcoxon signed-rank test [54] on per-case Final scores, with Holm correction within each family of comparisons [55], and McNemar's test [56] on exact entity and exact type. The analysis plan fixed five hypotheses before the LLM runs: **H1** the statistical core carries localisation ($|\mathrm{EL}(GAMA + \mathrm{LLM}) - \mathrm{EL}(GAMA)| \leq 3$ points for every backbone); **H2** the LLM's FI gain is bounded by the coverage of the statistical shortlist; **H3** the narrator's step-anchored checkpoint rate rises with model strength; **H4** structure beats scale (GAMA+LLM with the 7B model > Evidence→LLM with the large model); **H5** constant cost (≤ 7 LLM calls per incident). For Belief→LLM: **B1** does a free LLM change the computed decision, **B2** does the belief help the LLM (one-sided against Evidence→LLM), and **B3** free versus bounded use of the same LLM.

Status of RCA100. RCA100 was the development data of GAMA-RCA: earlier versions were designed and evaluated on all 103 cases, and the final modules were developed on the odd-numbered cases under a pre-registered protocol with a later run on the even-numbered half. Because per-case errors of earlier versions had been inspected on both halves, we report RCA100 as one dataset of 103 cases and make no held-out claim on it; generalisation is tested on AIOps2025 (RQ4). Every design change, its constants and its result were recorded in a dated pre-registration log before the corresponding run, and all runs are reported, including negative ones.

5.5. Implementation

GAMA-RCA is implemented in Python (pandas, NumPy, NetworkX) as a package with one switch per module; the only dataset-specific knowledge is a profile file (metric-name patterns, entity kinds, the status code of a failing span) and the data loader. A golden snapshot test fixes the outputs of every module configuration on synthetic incidents, and the refactored package was verified to reproduce the frozen development code on all 103 cases (206 of 206 configuration–case pairs identical, including every LLM prompt). CPU arms ran on Kaggle CPU sessions (4 cores), the local LLM on one T4 GPU.

6. Results

6.1. RQ1: accuracy of the statistical core

Table 4 and Fig. 2 report all arms on the 103 RCA100 cases. Without any language model, GAMA reaches a Final score of 62.9 [57.4, 68.5]; it names the exact root entity in 64 cases, the exact type in 45 and both in 39, in 78 s per incident on a CPU. An LLM that decides from the same calibrated evidence (Evidence→LLM, Qwen2.5-7B) reaches 42.7: the difference, +20.2 [15.0, 25.6] ($p = 2.8 \times 10^{-10}$; $\text{Final}_{\text{step}}$ +13.5, $p = 1.2 \times 10^{-6}$), comes from the type (exact type 33 cases for GAMA only versus 9 for the LLM only, McNemar $p = 0.0003$), not from the entity (8 versus 10, $p = 0.81$); given good evidence, the 7B model localises as well as the computed belief (EL 71.8 versus 68.9) but types far worse (FI 20.4 versus 43.7). An LLM that sees the alert only (Direct LLM) reaches 18.8 and never names the exact entity (GAMA − Direct: +44.1 [38.0, 50.3], $p = 1.5 \times 10^{-17}$). The evidence thus explains about half of the gap between the alert-only LLM and GAMA, and the computed belief the other half.

Table 4: Results on the 103 RCA100 cases (0–100; 95% bootstrap CI). Final uses the headline RP, $\text{Final}_{\text{step}}$ the step-anchored RP (Section 3). Exact: number of cases with the exact entity / type / both.

Arm	Final	$\text{Final}_{\text{step}}$	EL	FI	RP	Exact
GAMA (no LLM)	**62.9** [57.4, 68.5]	**56.2** [50.4, 61.9]	68.9	**43.7**	74.1	64/45/39
GAMA −M2	57.3 [52.0, 62.9]	50.6 [45.0, 56.5]	64.2	32.0	73.3	57/33/26
GAMA+LLM, 7B	59.2 [53.9, 64.6]	52.5 [47.0, 58.1]	66.5	35.0	73.7	60/36/30
GAMA+LLM, 120B	59.4 [54.0, 65.2]	52.7 [47.1, 58.6]	65.7	36.9	73.7	59/38/32
Belief→LLM, 7B	48.0 [42.6, 53.6]	48.0 [42.6, 53.6]	67.9	**43.7**	25.8	63/45/39
Evidence→LLM, 7B	42.7 [38.1, 47.4]	42.7 [38.1, 47.4]	**71.8**	20.4	26.2	66/21/18
Direct LLM, 7B	18.8 [15.3, 22.5]	18.8 [15.3, 22.5]	28.0	14.6	10.9	0/15/0
Evidence→LLM, 120B	**[Pending: run]**					
Belief→LLM, 120B	**[Pending: run]**					

7B = Qwen2.5-7B-Instruct (local); 120B = gpt-oss-120b (API).

By fault group. Figure 3 breaks EL and FI down by fault group. GAMA localises JVM-runtime faults perfectly and types 69% of them; resource and performance faults are typed in 80% of the cases. Its weak spot is node faults

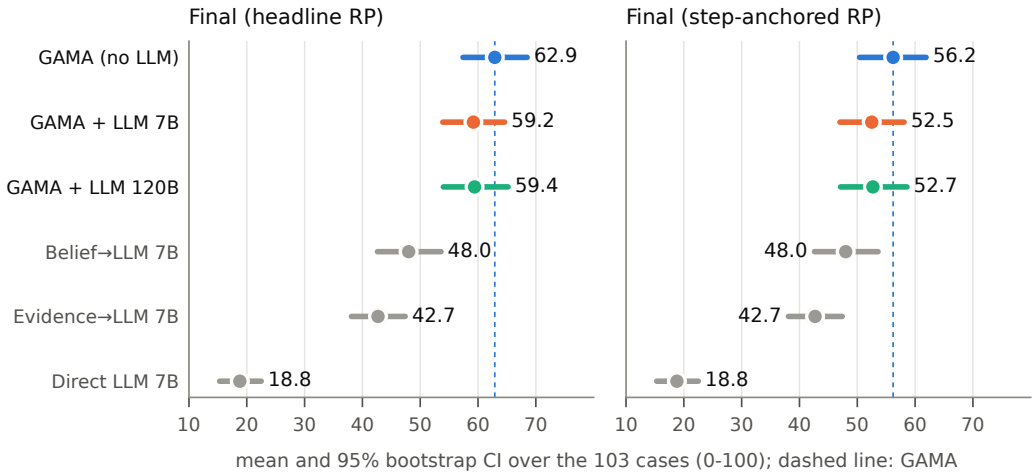


Figure 2: Final score of every arm on the 103 RCA100 cases with 95% bootstrap intervals; left with the headline reasoning-process score, right with the step-anchored one. The dashed line marks GAMA. Arms in which an LLM writes the chain return no evidence items, so the retrieval-based half of the headline RP is zero for them; the step-anchored score compares chains on equal terms.

(EL 22.3): a node fault slows every service it hosts, and when the node's own CPU or memory signal does not stand out against its peers, a hosted service with strong trace evidence wins. Middleware and database faults are localised (EL 77.9) but rarely typed (FI 23.1); plausibly because slow SQL and database network latency look alike in spans when no database-side metrics are exported.

6.2. RQ2: contribution of the modules

Table 5 reports the leave-one-module-out runs. Removing trace-origin attribution (M2) costs 5.6 Final points [2.3, 9.3] ($p = 0.009$; 18 cases better with the module, 5 worse): the module mainly identifies the *type* (exact type $14 : 2$, McNemar $p = 0.004$), because the messages and status codes of the origin spans name rate limiting, null pointers and HTTP 5xx errors (application-logic FI 39.5 with the module versus 10.5 without). It costs node faults, however: node EL drops from 33.9 to 22.3 with the module, since a node that hosts several origin services can lose to one of them, and a node without its own signal can win on their strength. **[Pending: leave-one-out runs of M1, M3, M4, M5 and M6.]**

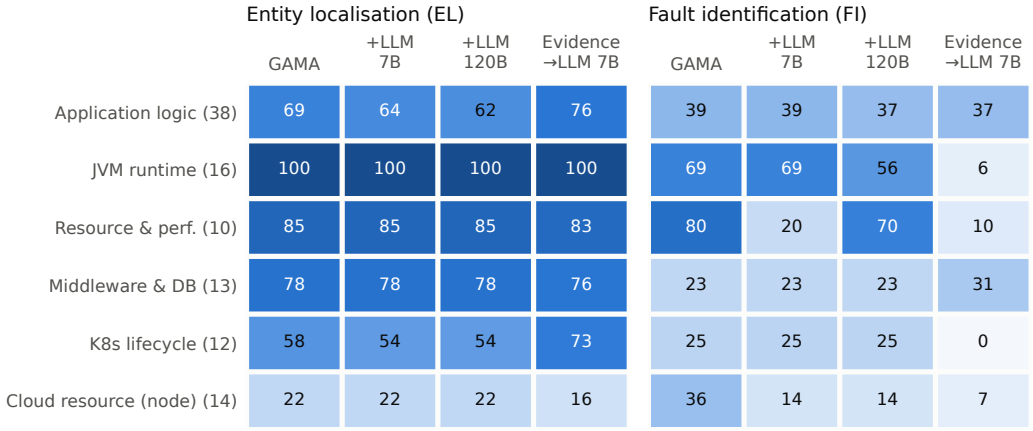


Figure 3: Entity localisation (left) and fault identification (right) by RCA100 fault group, for GAMA, GAMA with the bounded LLM (7B and 120B) and Evidence→LLM (7B). Darker is better.

Table 5: Leave-one-module-out ablation on the 103 RCA100 cases: change of the Final score when a module is removed from GAMA (paired Wilcoxon, two-sided, Holm over the six modules), cases better / worse with the module, and McNemar counts for the exact type (with : without).

Module removed	ΔFinal [95% CI]	p	better / worse	exact type
M1 corroboration	XX	XX	XX	XX
M2 trace origins	−5.6 [−9.3, −2.3]	0.009†	18 / 5	14 : 2
M3 mechanism typing	XX	XX	XX	XX
M4 load explain-away	XX	XX	XX	XX
M5 mechanism text	XX	XX	XX	XX
M6 cited chain (RP only)	XX	XX	XX	–

† unadjusted; the Holm adjustment is applied once all six runs are available.

6.3. RQ3: what the language model adds

Decision. With either backbone, the bounded LLM lowers the Final score of the statistical core (Table 6, Fig. 4a): by 3.7 points with Qwen2.5-7B ($p = 0.008$; 2 cases better, 14 worse) and by 3.5 points with gpt-oss-120b ($p = 0.03$; 3 better, 13 worse). The two backbones end at the same score (+0.2, $p = 0.62$). Two mechanisms are responsible (post-hoc analysis, no design change). (i) *Shortlist typing* changed the computed type in 25 (7B) and 21 (120B) cases; only 3 and 2 of these changes reached the true type, while 10 and 7 replaced a correct type (Fig. 4b). The changes concentrate where the telemetry cannot separate two types: the 7B model turned *cpuFullLoad* into *cpuDeadLoop* in seven cases; the 120B model avoided this pattern but turned *memoryPressure* into *fullGC* in two; both models replaced node CPU faults by disk or memory faults (node FI 35.7 → 14.3). (ii) The *text sensor* moved a correct entity away in four cases—the same four cases with both backbones—which points to the weight of the sensor's answer in the likelihood rather than to the model.

The computed belief and a free LLM. Given the posterior and full authority (Belief→LLM), the 7B model kept GAMA's entity and type in 102 of 103 cases (the remaining case is a truncated answer that also repeated it): it defers to the computed belief (B1). The belief helps the LLM: Belief→LLM exceeds Evidence→LLM by 5.3 points [0.2, 10.4] (one-sided $p = 0.025$; exact type 33 : 9), confirming B2. Compared with the bounded use of the same model (B3), the free LLM decides better (exact type 11 : 2), but its own chain is weaker than the computed one (chain-node match 51.5 versus 72.7, step-anchored checkpoints 0.0 versus 0.307), so its $\text{Final}_{\text{step}}$ is 4.4 points lower.

Explanation. Scale changes the explanation, not the decision (Fig. 4c). The narrator of gpt-oss-120b satisfies 0.284 of the step-anchored evidence checkpoints against 0.107 for the 7B narrator on the same cases (51 cases higher, 5 lower; one-sided Wilcoxon $p = 2 \times 10^{-10}$), close to the computed chain (0.307); 88% of the numbers it quotes appear in the evidence (7B: 92%). This supports H3.

6.4. Cost

GAMA needs no GPU and no model: 78 s per incident on four CPU cores, most of it reading the incident's files. The bounded LLM (Table 7) adds 2.7 calls and about 9,000 tokens per incident (7B: 181 s on a T4; 120B: 128 s

Table 6: Pre-registered comparisons and hypotheses on the 103 RCA100 cases. Δ is the paired difference of Final scores (first minus second) with 95% bootstrap CI; p from the Wilcoxon signed-rank test (Holm within family).

Comparison	ΔFinal [95% CI]	$\Delta\text{Final}_{\text{step}}$	p	Outcome
GAMA vs Evidence→LLM (7B)	+20.2 [15.0, 25.6]	+13.5	2.8×10^{-10}	structure wins
GAMA+LLM (7B) vs GAMA	−3.7 [−6.4, −1.4]	−3.7	0.008	LLM lowers the score
GAMA+LLM (120B) vs GAMA	−3.5 [−6.3, −0.9]	−3.5	0.03	LLM lowers the score
GAMA+LLM (120B) vs GAMA+LLM (7B)	+0.2 [−2.2, 2.6]	+0.2	0.62	no effect of scale
GAMA+LLM (7B) vs Evidence→LLM (7B)	+16.5 [11.8, 21.2]	+9.8	8.2×10^{-10}	structure wins
B2: Belief→LLM vs Evidence→LLM (7B)	+5.3 [0.2, 10.4]	+5.3	0.025	supported
B3: Belief→LLM vs GAMA+LLM (7B)	−11.2 [−13.8, −8.4]	−4.4	1.6×10^{-11}	better type, weaker chain
H1: $\lvert\Delta\text{EL}\rvert \leq 3$	7B: 2.4	120B: 3.2		met for 7B, narrowly not for 120B
H2: FI gain $\leq$ shortlist coverage	no FI gain with either backbone			met
H3: narrator checkpoints rise with scale	$0.107 \rightarrow 0.284$, $p = 2 \times 10^{-10}$			supported
H4: GAMA+LLM (7B) > Evidence→LLM (120B)	**[Pending: run]**			
H5: ≤ 7 LLM calls per incident	mean 2.7, max 5 (both backbones)			met

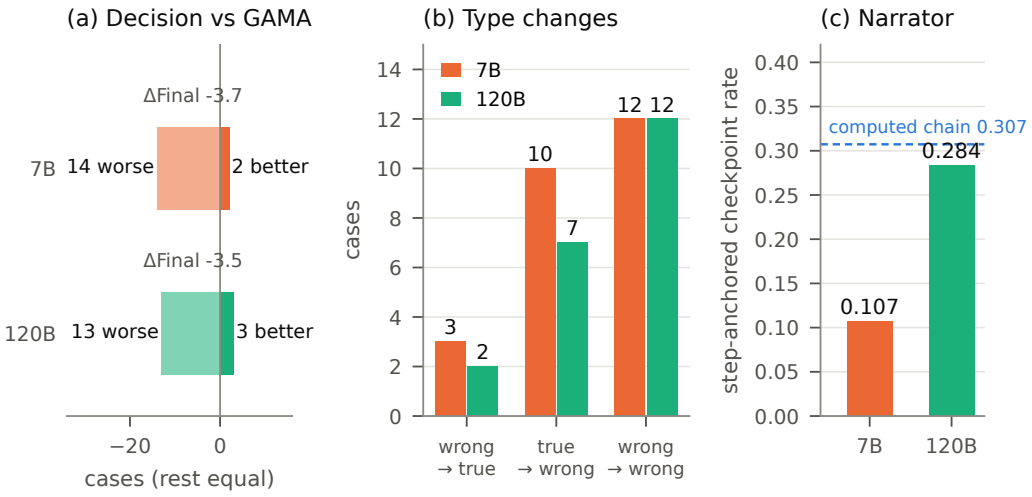


Figure 4: What the bounded LLM adds. (a) Cases in which GAMA + LLM scores better or worse than GAMA (the remaining cases are equal). (b) Changes of the computed type made by shortlist typing. (c) Step-anchored checkpoint rate of the LLM narrator; the dashed line is the chain computed by M6.

through the API, where 1,200 of the 1,400 output tokens are reasoning). The full 120B run of GAMA+LLM on 103 incidents cost about USD 0.40. For comparison, belief-state agents consume about 1.8×10^6 tokens per RCA100 episode [19], and open agents of the strongest configurations studied by Lu et al. about 1.5×10^6 [17]; their scores are not comparable with ours (different protocols and backbones), but the two-to-three orders of magnitude in cost are.

Table 7: Cost per incident on RCA100 (means over 103 cases; time includes reading the incident's files).

Arm	Hardware	s / case	LLM calls	tokens (input / output)
GAMA (no LLM)	CPU, 4 cores	78	0	0
GAMA+LLM, 7B	T4 GPU	181	2.7	9,157 (8,986 / 228)
GAMA+LLM, 120B	CPU + API	128	2.7	9,012 (7,592 / 1,421)
Belief→LLM, 7B	T4 GPU	152	1	4,812 (4,585 / 227)
Evidence→LLM, 7B	T4 GPU	168	1	4,337 (4,128 / 209)
Direct LLM, 7B	T4 GPU	38	1	983 (856 / 127)

6.5. RQ4: zero-shot transfer to AIOps2025

Protocol. The transfer was pre-registered before any run. What stays fixed is the method: the likelihood model, the roles, every constant, the decision and the modules. What may change is dataset knowledge written without AIOps2025 labels: (A1) an adapter from the release files to the canonical incident; (A2) metric-name patterns (e.g. `rrt` → latency) and the failing-span status; (A3) a topology derived from parent–child spans and from the published architecture (AIOps2025 ships no topology); (A4) pod entities with one rule—a service-level fault shows its mechanism in every pod, a pod-level fault in one pod while its siblings stay normal; (A5) fingerprints for the AIOps2025 fault types written from their published definitions; (A6) the benchmark's output format; (A7) trace sampling (all failing traces and a fixed hash-selected 20% of the others). A synthetic release in the real file layout tests the adapter (a CPU fault in one pod is located at that pod, in all pods at the service) and that ground-truth files are never opened. An implementation audit before the run found that the planned removal of other incidents' windows from the history would leave too little history for the frozen coverage rule (Eq. (2)); it was replaced, before any run, by the robust history statistics already in the method.

Results. **[Pending: AIOps2025: label-free checks (adapter coverage, testable and holding signals), then one run on the 400 cases; LA, TA, explainability, efficiency and the benchmark Final score with bootstrap CIs; breakdown by fault category and level (service / pod / node). Context, not a controlled comparison: the published accuracy of LLM agents on AIOps2025 [17].]**

7. Discussion

Why a computed belief. The comparison of Evidence→LLM, Belief→LLM and GAMA separates three ingredients: good evidence, a belief over hypotheses, and who decides. Calibrated evidence alone lets a 7B model localise as well as the statistical core, but not type the fault (FI 20.4 versus 43.7): choosing among 28 types requires weighing many weak signals against each other and against their background rates, which a likelihood does consistently and a language model does not. Once the posterior is given, the same model adopts it almost always. The belief is therefore the part that carries the diagnosis, and it can be computed. This complements belief-state

agents [19, 20]: their beliefs are written by the model at a cost of millions of tokens, ours is computed from calibrated likelihoods in seconds, and an LLM can still read it.

Where an LLM helps. Bounding the LLM did not make it a better decision maker: when the evidence does not separate two types, the model chooses the more specific one and loses more often than it wins, at 7B and at 120B alike. Scale changes the narrative instead—the larger model's chain cites the right values at the right steps almost as often as the computed chain. The practical design that follows is a separation of duties: the decision and its evidence are computed and auditable, and the language model explains them. The results also suggest how a later version could let an LLM contribute to the decision without the damage observed here: as a sensor whose answers are calibrated like any other categorical observation (against the incident's own background rate), or as a typing step that must defer to the posterior unless it quotes evidence that discriminates between the top types. Because these ideas were formed after seeing the RCA100 results, they must be evaluated on data not used for their design.

Failure modes ruled out by construction. Several reasoning failures that Lu et al. [17] observe in LLM agents cannot occur in GAMA-RCA by construction (Table C.10): every signal is tested against a baseline, every entity's resource signals are computed, timing enters as onset precedence, caller and callee roles are fixed by the trace structure and the load direction, and the whole hypothesis space is scored at once rather than anchored on the first idea. The remaining errors are mostly limits of the evidence—node faults without a distinctive node signal, database faults without database metrics—rather than of the reasoning.

Threats to validity. Internal. RCA100 was the development data; although each change was pre-registered and the final modules were developed on half of the cases, the 103-case results are optimistic, and we make no held-out claim on them. The fingerprint tables were written from the published fault definitions, but their additions were informed by development errors. *Construct.* Our scorer re-implements the published RCA100 protocol; the retrieval-based half of the headline RP is independent of the decision and favours systems that return evidence items, which is why every comparison is also reported with the step-anchored variant. The Final weights of the benchmark (0.4 / 0.3 / 0.3) are taken as given. *Statistical.* With 103 cases,

differences of about three points are at the edge of detectability; we report intervals and exact case counts with every test. API decoding is not deterministic; we report the single pre-registered run of each arm. *External.* RCA100 covers one application on one platform; the AIOps2025 transfer (Section 6.5) tests a second system with different fault types and pod-level faults. Two backbones do not span the space of LLMs; a stronger reasoning model or a different interface may behave differently.

Ethics and data. No personal data are involved. Both benchmarks are used under their non-commercial licences and are not redistributed. The API runs used a provider setting without data retention; prompts contain only benchmark telemetry.

8. Conclusion

GAMA-RCA computes the diagnosis of a microservice incident: calibrated multimodal evidence updates a Bayesian belief over (entity, fault type) hypotheses that explains symptoms away along the topology, estimates its propagation strength per incident and respects temporal precedence; a two-stage decision and an evidence-cited chain follow. Without labels, training or a language model, it reaches 62.9 Final on the 103 RCA100 cases on a CPU, 20.2 points above a 7B LLM that decides from the same evidence. In a pre-registered study, a bounded LLM did not improve the decision at 7B or at 120B parameters, while the larger model wrote a far better explanation. The decision should be computed and the language model should explain it. Future work covers the zero-shot AIOps2025 evaluation (Section 6.5), node faults without distinctive node signals, and LLM roles that enter the belief only as calibrated evidence, evaluated on data not used for their design.

CRediT authorship contribution statement

[Pending: roles of each author (Conceptualization, Methodology, Software, Validation, Writing – original draft, . . .).]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

The code (the `gama/` package, the evaluation scripts, the pre-registration documents and the synthetic tests) is available at `https://github.com/solaemanachmad/gamarca`. Every run writes a manifest (commit, modules, model, hardware, library versions, wall time), the scored results, the decision trail of every case and every LLM call (prompt, output, input and output tokens, latency); these artefacts are released with the code. RCA100 (CC BY-NC-SA) and AIOps2025 (CC BY-NC 4.0) are obtained from their publishers and are not redistributed.

Declaration of generative AI and AI-assisted technologies in the writing process

[Pending: Elsevier declaration: state which AI-assisted tools were used, for which tasks (e.g. code development, language editing), and that the authors reviewed and edited the content and take full responsibility for it.]

Acknowledgements

This research was supported by the Hibah DAMAS Penelitian 2026 of Sekolah Vokasi, Universitas Gadjah Mada.

Appendix A. Constants

Table A.8: Constants of GAMA-RCA, fixed a priori (none fitted to labels).

Symbol	Value	Meaning
α	0.05	level at which a calibrated signal holds; background of a continuous observation
B, K	4, 5	history length (window lengths) and number of pseudo-windows
coverage	0.8 / 0.5	minimum sample coverage of the history / the window
scale floors	$0.05\lvert\,\mathrm{med}\,\rvert$, 1	relative floor of the robust scale; floor of own-volatility and peer re-standardisation
peers, pool	≥ 3, ≥ 20	minimum peers; minimum null pool (else pooled over the entity kind)
onset	3 SD, 2 samples	onset threshold of a holding signal
θ_K	0.8	probability that some key signal holds at the true root
θ_U	0.6	support / may signal at the root
θ_P	0.6	own symptom at the root; load explained from upstream
α_C	0.05	contradicting signal at the root
categorical background	Beta, 10 pseudo-entities	shrinkage of the per-incident share towards α
$\Pr(h_0)$, fallback weight	0.1, 0.5	prior of no fault; prior weight of the two fallback types
π	$[0.06, 0.95]$, 30 iterations	EM bounds and iteration limit for the propagation strength
θ_T, tolerance	0.9, 60 s	temporal precedence (onsets and intra-entity order)
θ_M, β_M floor, weight	0.6, 0.1, 10	cross-modal corroboration (M1)
trace origins	30%, 10 / 5 traces	error origin share and minimum error / slow traces (M2)
chain	4 values per step	window values cited per chain step (M6)
sensor	$\Pr \geq 0.8$, 0.05 bits, 5 probes	stop rule, minimum information gain, budget (M7)
typing	top 5, quote ≥ 10 chars	shortlist size and quote length (M7)
decoding	temp. 0, seed 0	greedy decoding; 1,024 new tokens (local), 8,192 incl. reasoning (API)

Appendix B. Fingerprint examples

Table B.9: Excerpt of the RCA100 fingerprint table (roles of signals per fault type; "a&b" requires both). The full table and the AIOps2025 table are in the code.

Type	Key (one of)	Support	Contradiction
nodeCpuHigh	node CPU rise	–	–
nodeDown	node NotReady event; node ready drop	node pods drop	scale-down event
cpuFullLoad	CPU rise	latency rise	request rise
memoryPressure	memory rise	GC rise, CPU rise	–
fullGC	GC rise	GC log rise, GC message, latency rise	–
rateLimiting	HTTP 429; rate-limit message or log rise	HTTP 403	–
trafficSurge	request rise	latency rise, CPU rise	–
replicaScaleDown	scale-down event; replica / availability drop; unavailable	HTTP 503, connection refused, request drop	crash-loop, unschedulable
redisUnavailable	failing Redis spans; Redis message	connection refused, timeout	–
slowSQL	slow database spans	latency rise, SQL message	failing database spans
httpError5xx (fallback)	HTTP 5xx	failing spans, error rise	–

Appendix C. Agent failure modes ruled out by construction

Table C.10: Failure modes of LLM agents catalogued by Lu et al. [17] and the mechanism of GAMA-RCA that prevents each.

Failure mode	Mechanism in GAMA-RCA
Baseline gap (no comparison with normal behaviour)	every signal is scored against the incident's own history (stage 1)
Edge untested	propagation is explained only along topology edges; trace origins test caller–callee spans (M2)
Resource gap	the resource signals of every entity are always computed; there is no query choice
Ranking gap	peer re-standardisation rejects shifts common to all entities of a kind
Timing gap	onset precedence (stage 2) and intra-entity signal order (M3)
Silence taken as health	availability and replica drops are observations
Salience bias (chronically noisy entity)	own-volatility re-standardisation; EM penalises wide hypotheses
Caller / callee semantics reversed	trace origin with callee relay (M2); directional load explain-away (M4)
Anchor lock	no sequential anchoring: all hypotheses are scored at once
Schema-out claim (type outside the taxonomy)	typed hypothesis space; the LLM chooses only within the shortlist (M7)
Friction stop	fixed pipeline without a tool loop

References

[1] A. Avižienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, IEEE Transactions on Dependable and Secure Computing 1 (1) (2004) 11–33.

[2] J. Soldani, A. Brogi, Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey, ACM Computing Surveys 55 (3) (2022).

[3] S. Zhang, et al., Failure diagnosis in microservice systems: A comprehensive survey and analysis, ACM Transactions on Software Engineering and Methodology 35 (1) (2025).

[4] L. Fu, et al., Intelligent root cause localization in microservice systems: A survey and new perspectives, ACM Computing Surveys 57 (12) (2025).

[5] Y. Cai, et al., A multi-dataset benchmark for evaluating LLM agents in microservice failure diagnosis, arXiv preprint arXiv:2606.29193 (2026).

[6] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020.

[7] A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, M. Kocaoglu, Root cause analysis of failures in microservices through causal discovery, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 31158–31170.

[8] L. Pham, H. Ha, H. Zhang, BARO: Robust root cause analysis for microservices via multivariate Bayesian online change point detection, Proceedings of the ACM on Software Engineering 1 (FSE) (2024) 2214–2237.

[9] M. Li, et al., Causal inference-based root cause analysis for online service systems with intervention recognition, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2022, pp. 3230–3240.

[10] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, Z. Zheng, Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2023, pp. 553–565.

[11] L. Zheng, Z. Chen, J. He, H. Chen, MULAN: Multi-modal causal structure learning and root cause analysis for microservice systems, in: Proceedings of the ACM Web Conference (WWW), 2024, pp. 4107–4116.

[12] Y. Chen, et al., Automatic root cause analysis via large language models for cloud incidents, in: Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys), 2024, pp. 674–688.

[13] Z. Wang, et al., RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models, in: Proceedings of

the 33rd ACM International Conference on Information and Knowledge Management (CIKM), 2024.

[14] J. Xu, et al., OpenRCA: Can large language models locate the root cause of software failures?, in: The Thirteenth International Conference on Learning Representations (ICLR), 2025.

[15] C. Pei, et al., Flow-of-action: SOP enhanced LLM-based multi-agent system for root cause analysis, in: Companion Proceedings of the ACM on Web Conference 2025, 2025, pp. 422–431.

[16] W. Zhang, et al., mABC: Multi-agent blockchain-inspired collaboration for root cause analysis in micro-services architecture, in: Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 4017–4033.

[17] X. Lu, et al., Beyond fault localization: A trajectory-level study of LLM agents for microservice root cause analysis, arXiv preprint arXiv:2608.21310 (2026).

[18] M. Riddell, et al., Stalled, biased, and confused: Uncovering reasoning failures in LLMs for cloud-based root cause analysis, in: Proceedings of FORGE, 2026.

[19] X. Luo, et al., Beyond memory: Harnessing long-horizon agents with explicit belief states, arXiv preprint arXiv:2610.01415 (2026).

[20] X. Luo, et al., Graph of states: Solving abductive tasks with large language models, arXiv preprint arXiv:2603.21250 (2026).

[21] J. Lin, P. Chen, Z. Zheng, Microscope: Pinpoint performance issues with causal graphs in micro-service environments, in: International Conference on Service-Oriented Computing (ICSOC), 2018, pp. 3–20.

[22] D. Liu, et al., MicroHECL: High-efficient root cause localization in large-scale microservice systems, in: IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2021.

[23] Y. Meng, et al., Localizing failure root causes in a microservice through causality inference, in: IEEE/ACM 28th International Symposium on Quality of Service (IWQoS), 2020, pp. 1–10.

[24] H. Shan, et al., ϵ-diagnosis: Unsupervised and real-time diagnosis of small-window long-tail latency in large-scale microservice platforms, in: The World Wide Web Conference (WWW), 2019.

[25] Z. Li, et al., Practical root cause localization for microservice systems via trace analysis, in: IEEE/ACM 29th International Symposium on Quality of Service (IWQoS), 2021, pp. 1–10.

[26] G. Yu, et al., MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments, in: Proceedings of the Web Conference (WWW), 2021.

[27] X. Guo, et al., Graph-based trace analysis for microservice architecture understanding and problem diagnosis, in: Proceedings of ESEC/FSE, 2020.

[28] S. Zhang, et al., Robust failure diagnosis of microservice system through multimodal data, IEEE Transactions on Services Computing 16 (6) (2023).

[29] C. Lee, T. Yang, Z. Chen, Y. Su, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023.

[30] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann, 1988.

[31] P. Bahl, R. Chandra, A. Greenberg, S. Kandula, D. A. Maltz, M. Zhang, Towards highly reliable enterprise network services via inference of multi-level dependencies, in: Proceedings of ACM SIGCOMM, 2007.

[32] I. Rish, et al., Adaptive diagnosis in distributed systems, IEEE Transactions on Neural Networks 16 (5) (2005).

[33] D. V. Lindley, On a measure of the information provided by an experiment, The Annals of Mathematical Statistics 27 (4) (1956) 986–1005.

[34] B. Efron, Large-scale simultaneous hypothesis testing: The choice of a null hypothesis, Journal of the American Statistical Association 99 (465) (2004) 96–104.

[35] V. Vovk, A. Gammerman, G. Shafer, Algorithmic Learning in a Random World, Springer, 2005.

[36] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[37] D. Roy, et al., Exploring LLM-based agents for root cause analysis, in: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion), 2024.

[38] Y. Tian, Y. Liu, Z. Chong, Z. Huang, H.-A. Jacobsen, GALA: Can graph-augmented large language model agentic workflows elevate root cause analysis?, arXiv preprint arXiv:2508.12472 (2025).

[39] L. Pham, H. Zhang, H. Ha, F. Salim, X. Zhang, RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data, in: Companion Proceedings of the ACM on Web Conference 2025, 2025, pp. 777–780.

[40] Y. Chen, et al., AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds, Proceedings of Machine Learning and Systems 7 (2025).

[41] A. Fang, et al., OpenRCA 2.0: From outcome labels to causal process supervision, arXiv preprint arXiv:2606.27154 (2026).

[42] Y. Han, et al., The potential of one-shot failure root cause analysis: Collaboration of the large language model and small classifier, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2024.

[43] D. Zhang, et al., LM-PACE: Confidence estimation by large language models for effective root causing of cloud incidents, in: Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion), 2024.

[44] V. Venktesh, M. Rathee, A. Anand, Trust but verify! A survey on verification design for test-time scaling, arXiv preprint arXiv:2508.16665 (2025).

[45] T. Gao, H. Yen, J. Yu, D. Chen, Enabling large language models to generate text with citations, in: Proceedings of EMNLP, 2023.

[46] L. P. Kaelbling, M. L. Littman, A. R. Cassandra, Planning and acting in partially observable stochastic domains, Artificial Intelligence 101 (1–2) (1998) 99–134.

[47] W. Tang, et al., Tru-POMDP: Task planning under uncertainty via tree of hypotheses and open-ended POMDPs, in: Advances in Neural Information Processing Systems, Vol. 38, 2026.

[48] A. P. Dempster, N. M. Laird, D. B. Rubin, Maximum likelihood from incomplete data via the EM algorithm, Journal of the Royal Statistical Society: Series B 39 (1) (1977) 1–38.

[49] B. Beyer, C. Jones, J. Petoff, N. R. Murphy, Site Reliability Engineering: How Google Runs Production Systems, O'Reilly Media, 2016.

[50] Chaos Mesh Authors, Chaos mesh: A chaos engineering platform for Kubernetes, `https://chaos-mesh.org` (2020).

[51] Qwen Team, Qwen2.5 technical report, arXiv preprint arXiv:2412.15115 (2024).

[52] OpenAI, gpt-oss-120b and gpt-oss-20b model card, `https://openai.com/index/introducing-gpt-oss/` (2025).

[53] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[54] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[55] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[56] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (2) (1947) 153–157.